Jose Fabian

CS 499: Computer Science Capstone

Milestone Two: Enhancement One — Software Design and Engineering Narrative

Artifact Description

The artifact for this enhancement is Travlr Getaways, a full-stack trip-booking platform I originally built using Express, Mongoose, and Angular as part of earlier web development coursework. The specific files enhanced in this milestone are trips.js and authentication.js, both located in the app_api/controllers directory. trips.js is the controller responsible for all CRUD operations on the Trip resource, including listing trips, retrieving a single trip, and creating, updating, and deleting trips. authentication.js handles user registration and login using Passport's local strategy and JSON Web Tokens.

Justification for Inclusion

I selected Travlr Getaways for the software design and engineering category because it is a complete, multi-layered application rather than an isolated script, which let me demonstrate architectural and design decisions across an entire request lifecycle rather than a single function in isolation. The controllers I enhanced showcase my ability to identify and correct real, practical software engineering problems: inconsistent error handling, missing input validation, incorrect HTTP status codes, and duplicated logic. These are exactly the kinds of issues a code reviewer or team lead would flag in a professional setting, and fixing them required me to think about maintainability and correctness, not just getting the code to run.

The artifact was improved in several concrete ways. In trips.js, I standardized every function on async/await with consistent try/catch error handling, replacing a mix of promise chains and unguarded await calls that left some functions with no error handling at all. I extracted the trip object mapping logic, which had been duplicated word for word between the create and update functions, into a single reusable buildTripFromBody function. I added real input validation through a validateTripBody function that checks required fields, types, and value ranges before any database operation runs. I corrected the update endpoint, which previously returned a 201 Created status on a successful update, to return the correct 200 status, and switched the single-trip lookup from Trip.find to Trip.findOne so its behavior matches its name and intent. Finally, I replaced every instance of returning a raw error object to the client with a sanitized message, while still logging the actual error server-side for debugging.

In authentication.js, I added real validation for email format and password strength, rather than only checking that fields were present. I added an explicit check for an existing account with the same email before attempting to save a new user, returning a clean 409 Conflict response instead of letting a database-level duplicate-key error leak back to the client. I corrected the status code returned when Passport itself encounters an authentication error, changing it from 404 to 500, since a 404 implies a missing resource rather than a server-side failure. I also added a rate limiter on the login route using express-rate-limit, capping repeated login attempts from the same IP address, since the original controller had no protection against brute-force password guessing at all.

Course Outcomes

In my Module One ePortfolio plan, I identified this artifact as demonstrating skill in software design and engineering, specifically architectural refactoring and the use of well-founded, industry-standard techniques and tools. This milestone met that goal: the refactor into consistent async/await patterns, the extraction of shared logic into a reusable function, and the addition of structured input validation all reflect deliberate, well-founded software engineering practice rather than a quick patch.

This work also ended up demonstrating meaningful progress toward the security mindset outcome, more than I had originally planned for this specific category. The authentication.js changes, in particular the duplicate-email handling, sanitized error responses, and login rate limiting, directly anticipate and mitigate real adversarial risks such as information leakage through error messages and brute-force credential attacks. I do not have any changes to my outcome-coverage plan overall, but I would now describe this artifact as touching both the software engineering outcome and the security mindset outcome, rather than software engineering alone.

Reflection

The most valuable part of this process was seeing how much a code review conducted with a checklist in mind, rather than a general read-through, surfaces problems that are easy to miss otherwise. When I originally wrote trips.js and authentication.js, the code worked, and that was enough at the time. Going back through it deliberately looking for inconsistent error handling, missing validation, and incorrect status codes made me realize how many small but real defects had shipped without me noticing, simply because nothing crashed during normal use.

The biggest challenge was deciding how far to take the input validation without introducing a new dependency I could not fully verify against the rest of the project's existing package configuration. I considered using a library like Joi or express-validator, which are common in production Node applications, but chose to write manual validation functions instead so the enhancement would not risk breaking the build if those packages were not already part of the project. This was a useful design tradeoff to work through directly: manual validation is more code to maintain, but it keeps the enhancement self-contained and dependency-safe, which mattered more given the scope of this milestone. If I were extending this further for production use, introducing a schema validation library would likely be the next step, since it would centralize the validation rules and reduce the amount of hand-written conditional logic.